



API SECURITY RISK ANALYSIS REPORT

API Endpoint Analysis | OWASP API Top 10 | CVE & MITRE ATT&CK Mapping

Prepared By	Satheesh Nithiananthan (CyberLycan)
Organisation	SAIZERO — Ground Zero Defence
Report Date	March 2026
APIs Tested	OWASP crAPI (localhost:8888)
Assessment Type	Read-Only API Security Risk Analysis
Methodology	OWASP API Security Top 10 (2023 Edition)
Tools Used	Postman, Browser DevTools, jwt.io, Mailhog
Scope	Public/Demo API — Educational Purposes Only
Classification	For Training Purposes Only
Internship	Cybersecurity Internship 2026

Table of Contents

1. Executive Summary	03
2. Assessment Scope & Methodology	04
Target API	
Methodology	
Tools Used	
3. OWASP API Security Top 10 — Overview	05
4. Detailed Findings	06
Finding 01 — JWT Token — Role in Payload & MFA Not Enforced	06-07
Finding 02 — Excessive Data Exposure	08-09
Finding 03 — BOLA — Vehicle GPS Location Exposed	10-11
Finding 04 — No Rate Limiting on Authentication Endpoint	12-13
Finding 05 — MFA Not Required — Single Factor Authentication Only	14-15
Finding 06 — Missing Content-Security-Policy Header	16-17
Finding 07 — CORS Wildcard Misconfiguration	18-19
Finding 08 — Server Version Disclosed in Response Headers	20-21
Finding 09 — Weak Email Change Token — No Expiry	22-23
5. Risk Classification Summary	24
6. Prevention Guidelines	24
For API Developers	
For Security Teams	
7. Conclusion	25

1. EXECUTIVE SUMMARY

SAIZERO — Ground Zero Defence was engaged to conduct a read-only API Security Risk Analysis against the OWASP crAPI platform as part of my Cybersecurity Internship 2026. The assessment focused on identifying API-specific vulnerabilities mapped to the OWASP API Security Top 10 (2023 Edition), with additional mapping to CVE references and MITRE ATT&CK framework techniques.

The assessment identified 9 confirmed vulnerabilities across 4 OWASP API Security categories. The most critical finding — Broken Object Level Authorization (BOLA) — allows any authenticated user to retrieve the real-time GPS location, full name, and email address of any other user in the system by manipulating vehicle UUIDs in API requests. This constitutes a physical safety risk beyond typical data breach scenarios.

Additional high-severity findings include absence of rate limiting on the authentication endpoint, CORS wildcard misconfiguration allowing cross-origin API access from any website, missing Content-Security-Policy headers, no multi-factor authentication enforcement, and a weak email change token with no expiry time.

Risk Level	Count	Findings
CRITICAL	1	BOLA — GPS Location Leaked
HIGH	6	Rate Limiting, MFA, CSP, CORS, Email Token, JWT
MEDIUM	1	Excessive Data Exposure
LOW	1	Server Version Disclosed
TOTAL	9	Findings confirmed across 4 OWASP API categories

2. ASSESSMENT SCOPE & METHODOLOGY

Target API

Application	OWASP crAPI (Completely Ridiculous API)
Base URL	http://localhost:8888
Services	/identity, /community, /workshop
Auth Method	JWT Bearer Token (RS256)
Test Accounts	usera@test.com, userb@test.com
Type	Read-Only — no exploitation or DoS testing

Methodology

- Endpoint discovery via API documentation and browser DevTools Network tab
- Authentication testing — JWT token analysis, brute force protection, MFA enforcement
- Authorization testing — BOLA via object ID manipulation across user accounts
- Data exposure analysis — response field review against minimum necessary principle
- Security header inspection on all endpoints via Postman response headers
- CORS policy testing via forged Origin headers
- Email flow analysis via Mailhog — token strength and expiry testing

Tools Used

Tool	Purpose
Postman	API request crafting, response analysis, header inspection
Browser DevTools	Network traffic monitoring, endpoint discovery, cookie analysis
jwt.io	JWT token decoding and algorithm analysis
Mailhog	Email token capture, expiry verification
OWASP crAPI	Target application — intentionally vulnerable API platform

3. OWASP API SECURITY TOP 10 — OVERVIEW

The OWASP API Security Top 10 is a separate framework from the standard OWASP Top 10 for web applications. It specifically addresses risks unique to API-based architectures including mobile backends, SaaS platforms, and microservices.

ID	Name	Found in this Assessment
API1:2023	Broken Object Level Authorization	YES — Finding 3 (CRITICAL)
API2:2023	Broken Authentication	YES — Findings 1, 4, 5, 9 (HIGH)
API3:2023	Broken Object Property Level Auth	YES — Finding 2 (MEDIUM)
API4:2023	Unrestricted Resource Consumption	YES — Finding 4 (HIGH)
API5:2023	Broken Function Level Authorization	Not tested in scope
API6:2023	Unrestricted Access to Sensitive Flows	Not tested in scope
API7:2023	Server Side Request Forgery	Not tested in scope
API8:2023	Security Misconfiguration	YES — Findings 6, 7, 8 (HIGH/LOW)
API9:2023	Improper Inventory Management	Not tested in scope
API10:2023	Unsafe Consumption of APIs	Not tested in scope

4. DETAILED FINDINGS

FINDING 01 — JWT TOKEN — ROLE IN PAYLOAD & MFA NOT ENFORCED

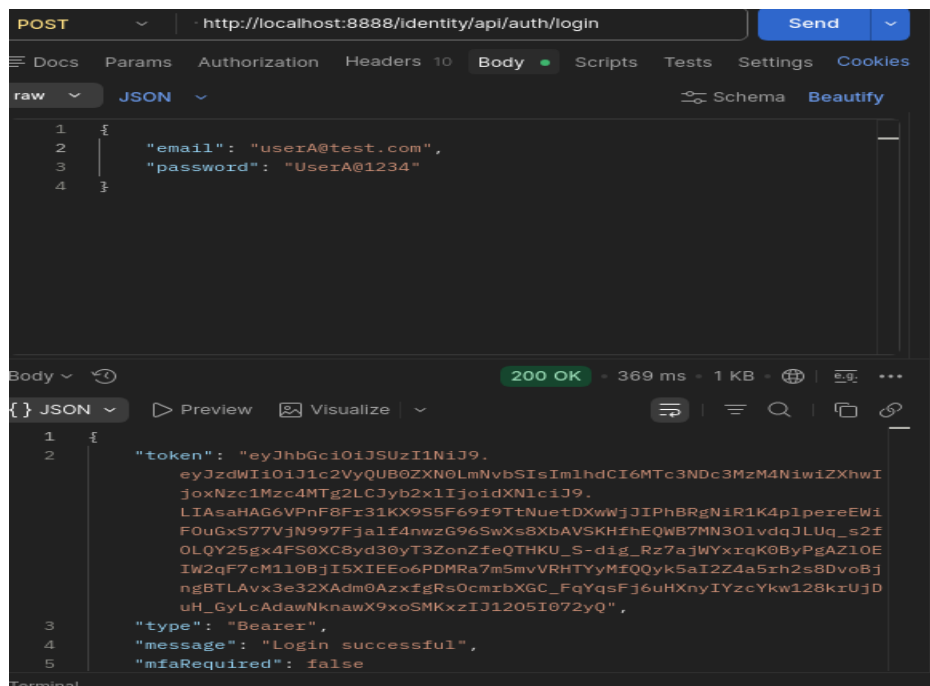
API2:2023 — Broken Authentication

Endpoint	POST /identity/api/auth/login
Method	POST
OWASP API	API2:2023 — Broken Authentication
CVE Reference	CVE-2022-21449 (ECDSA Signature Bypass) CWE-287 (Improper Authentication)
MITRE ATT&CK	T1552.004 — Credentials in Files T1078 — Valid Accounts
Severity	HIGH

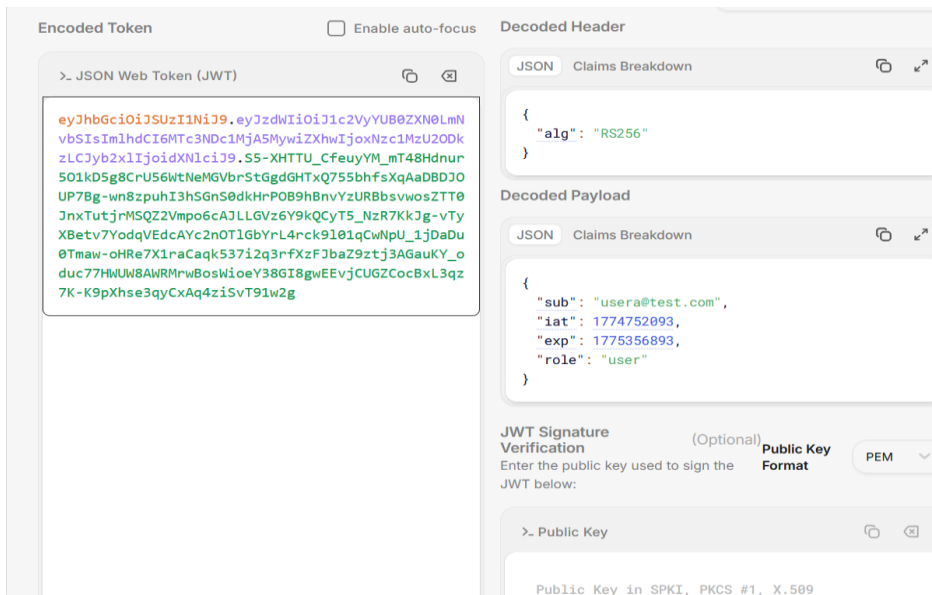
Description

The login endpoint returns a JWT token signed with RS256. While RS256 is a strong algorithm, the token payload contains the user role field which could be targeted for manipulation. Additionally the login response explicitly states mfaRequired: false indicating no multi-factor authentication is enforced for any user account.

Evidence



```
POST /identity/api/auth/login
Response 200 OK:
{ "token": "eyJhbGciOiJIUzI1NiJ9...", "type": "Bearer",
  "message": "Login successful", "mfaRequired": false }
```



Decoded JWT Payload:

```
{ "sub": "usera@test.com", "iat": 1774752093,
  "exp": 1775356893, "role": "user" }
```

CVE & MITRE Mapping

CVE-2022-21449	Psychic Signatures — Java ECDSA signature bypass. Demonstrates risks of improper JWT signature verification. Servers that fail to verify JWT signatures properly allow forged tokens.
CWE-287	Improper Authentication — weakness in verifying claimed identity. mfaRequired:false confirms single-factor authentication only.
T1552.004	MITRE ATT&CK: Credentials in Files — role stored in JWT payload could be extracted by any party who intercepts the token.
T1078	MITRE ATT&CK: Valid Accounts — no MFA means valid credentials provide immediate full access with no secondary barrier.

Business Impact

No MFA means a single compromised password gives attacker full account access. JWT role in payload is a stepping stone for privilege escalation if signature verification is ever weakened.

Remediation

- Enforce MFA for all user accounts — especially those with access to financial or location data
- Remove role from JWT payload — verify permissions server-side on each request
- Implement token rotation and short expiry windows (1 hour maximum)
- Monitor for anomalous login patterns across geographic locations

FINDING 02 — EXCESSIVE DATA EXPOSURE**API3:2023 — Broken Object Property Level Authorization**

Endpoint	GET /identity/api/v2/user/dashboard
Method	GET
OWASP API	API3:2023 — Broken Object Property Level Authorization
CVE Reference	CVE-2019-14234 (Django Data Exposure) CWE-213 (Exposure of Sensitive Information)
MITRE ATT&CK	T1530 — Data from Cloud Storage T1213 — Data from Information Repositories
Severity	MEDIUM

Description

The user dashboard endpoint returns 10 data fields when a typical user interface requires only 2-3 (name and email). Fields returned include internal database ID, phone number (PII), available credit balance (financial data), internal video reference IDs, and the internal role naming convention.

Evidence

The screenshot shows a REST client interface with a GET request to `http://localhost:8888/identity/api/v2/user/dashboard` using a Bearer token. The response is a 200 OK status with a JSON body containing 10 fields: `id`, `name`, `email`, `number`, `picture_url`, `video_url`, `video_name`, `available_credit`, `video_id`, and `role`.

```
{
  "id": 8,
  "name": "usera",
  "email": "usera@test.com",
  "number": "1234567890",
  "picture_url": null,
  "video_url": null,
  "video_name": null,
  "available_credit": 100.0,
  "video_id": 0,
  "role": "ROLE_USER"
}
```

GET /identity/api/v2/user/dashboard

Response: { "id": 8, "name": "usera", "email": "usera@test.com",
"number": "1234567890", "picture_url": null, "video_url": null,
"video_name": null, "available_credit": 100.0,
"video_id": 0, "role": "ROLE_USER" }

CVE & MITRE Mapping

CVE-2019-14234	Django QuerySet exposure — demonstrates how ORM objects returned directly to API responses expose internal fields not intended for users.
CWE-213	Exposure of Sensitive Information Due to Incompatible Policies — API returns PII and financial data without filtering.
T1530	MITRE ATT&CK: Data from Cloud Storage — attacker harvests user PII and financial data from API responses at scale.
T1213	MITRE ATT&CK: Data from Information Repositories — internal IDs and role names reveal system architecture.

Remediation

- Implement response filtering — return only fields necessary for the specific use case
- Use Data Transfer Objects (DTOs) to define exactly what each endpoint returns
- Never return internal database IDs or role naming conventions to frontend
- Separate financial data endpoints with additional authentication requirements

FINDING 03 — BOLA — VEHICLE GPS LOCATION EXPOSED**API1:2023 — Broken Object Level Authorization**

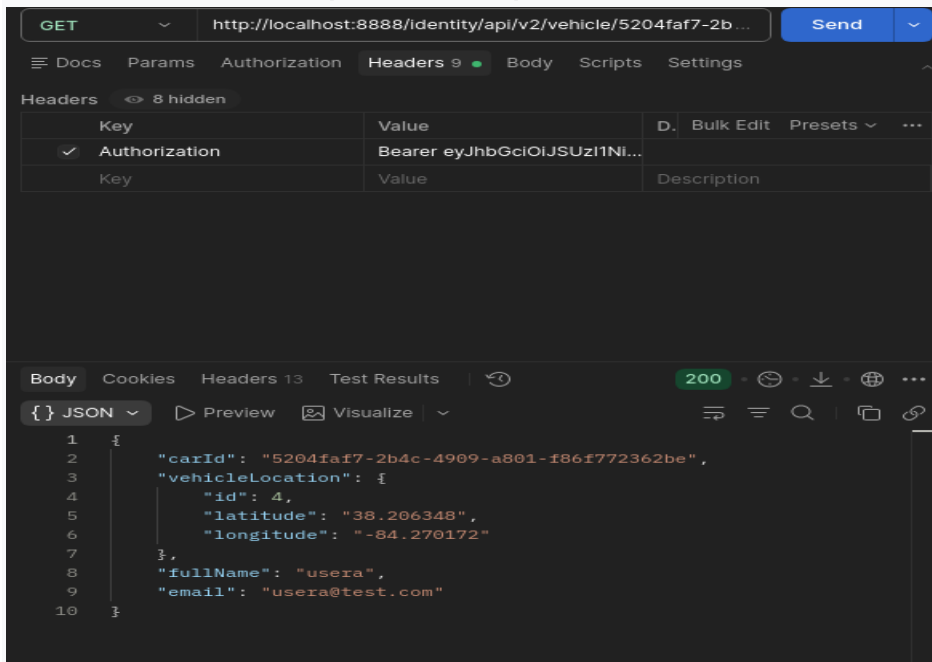
Endpoint	GET /identity/api/v2/vehicle/{uuid}/location
Method	GET
OWASP API	API1:2023 — Broken Object Level Authorization (BOLA)
CVE Reference	CVE-2021-20114 (TCEXAM IDOR) CWE-639 (Authorization Bypass Through User-Controlled Key)
MITRE ATT&CK	T1083 — File and Directory Discovery T1530 — Data from Cloud Storage
Severity	CRITICAL

Description

The vehicle location endpoint accepts a vehicle UUID and returns GPS coordinates, full name, and email of the vehicle owner. No ownership verification is performed — any authenticated user can retrieve location data for any vehicle by supplying its UUID. This constitutes a physical safety risk beyond typical data breach scenarios.

Evidence

GET /identity/api/v2/vehicle/5204faf7-2b4c-4909-a801-f86f772362be/location
Authorization: Bearer [User A Token]



Response 200 OK:

```
{ "carId": "5204faf7-2b4c-4909-a801-f86f772362be",  "vehicleLocation": { "id": 4,    "latitude": "38.206348", "longitude": "-84.270172" },  "fullName": "usera", "email": "usera@test.com" }
```

CVE & MITRE Mapping

CVE-2021-20114	TCEXam IDOR vulnerability — authenticated users access other users objects by manipulating IDs. Direct parallel to this finding.
CWE-639	Authorization Bypass Through User-Controlled Key — vehicle UUID is user-controlled and not validated against ownership.
T1083	MITRE ATT&CK: File and Directory Discovery — attacker enumerates vehicle UUIDs to discover and track multiple victims.
T1530	MITRE ATT&CK: Data from Cloud Storage — real-time GPS coordinates harvested at scale for target tracking.

Business Impact

CRITICAL PHYSICAL SAFETY RISK: Attacker can track victim location in real time, determine home and work addresses, monitor movement patterns, and plan physical attacks or burglary. This exceeds typical data breach impact.

Remediation

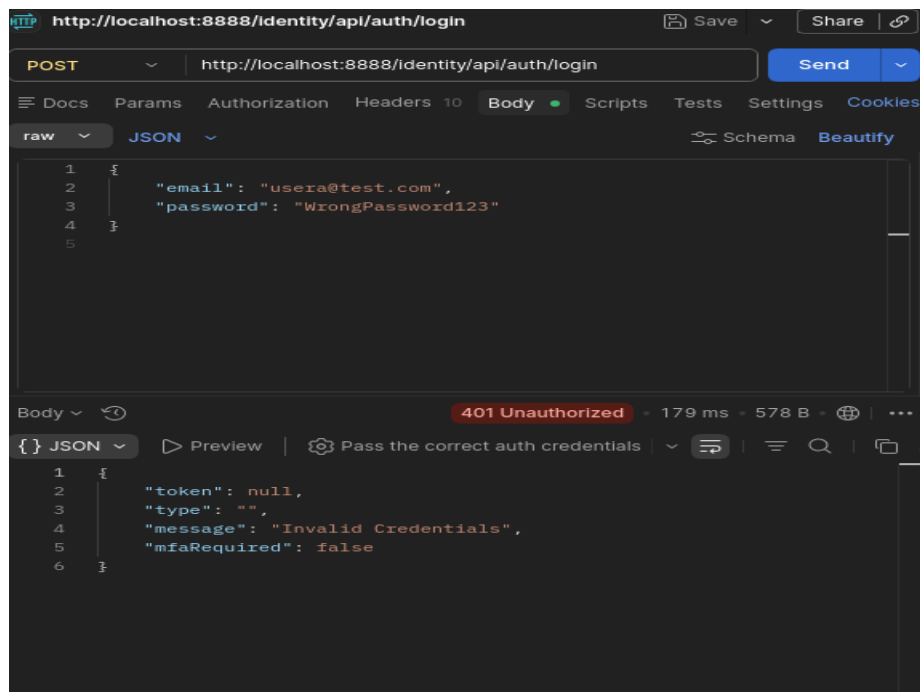
- Implement server-side ownership check on every vehicle location request
- Verify requesting user ID matches vehicle owner ID before returning data
- Return 403 Forbidden with no data disclosure when ownership check fails
- Implement UUID enumeration protection — rate limit and alert on sequential UUID attempts
- Log all vehicle location access with requesting user identity for audit trail

FINDING 04 — NO RATE LIMITING ON AUTHENTICATION ENDPOINT**API4:2023 — Unrestricted Resource Consumption**

Endpoint	POST /identity/api/auth/login
Method	POST
OWASP API	API4:2023 — Unrestricted Resource Consumption
CVE Reference	CVE-2019-19781 (Citrix Brute Force) CWE-307 (Improper Restriction of Excessive Auth Attempts)
MITRE ATT&CK	T1110 — Brute Force T1110.001 — Password Guessing
Severity	HIGH

Description

The login endpoint accepts unlimited authentication attempts without rate limiting, account lockout, or CAPTCHA. Ten consecutive failed login attempts were made with incorrect passwords — all returned 401 Unauthorized immediately with no throttling. Combined with no MFA (Finding 1), successful brute force leads directly to full account compromise.

Evidence

```
POST /identity/api/auth/login - Attempt 1
{ "password": "WrongPassword1" } → 401 Unauthorized (179ms)
POST /identity/api/auth/login - Attempt 10
{ "password": "WrongPassword10" } → 401 Unauthorized (179ms)
No rate limiting. No lockout. No CAPTCHA. No delay.
```

CVE & MITRE Mapping

CVE-2019-19781	Citrix ADC brute force — lack of rate limiting on auth endpoint allowed credential stuffing attacks affecting thousands of organisations.
CWE-307	Improper Restriction of Excessive Authentication Attempts — no mechanism to detect or respond to repeated failed logins.
T1110	MITRE ATT&CK: Brute Force — attacker systematically tries passwords against known email addresses.
T1110.001	MITRE ATT&CK: Password Guessing — common password lists used against user accounts with no detection or blocking.

Remediation

- Implement rate limiting — maximum 5 failed attempts per account per 15 minutes
- Return 429 Too Many Requests after threshold is reached
- Implement progressive delays — each failed attempt increases response time
- Lock account temporarily after repeated failures — notify user via email
- Deploy CAPTCHA after 3 consecutive failed attempts

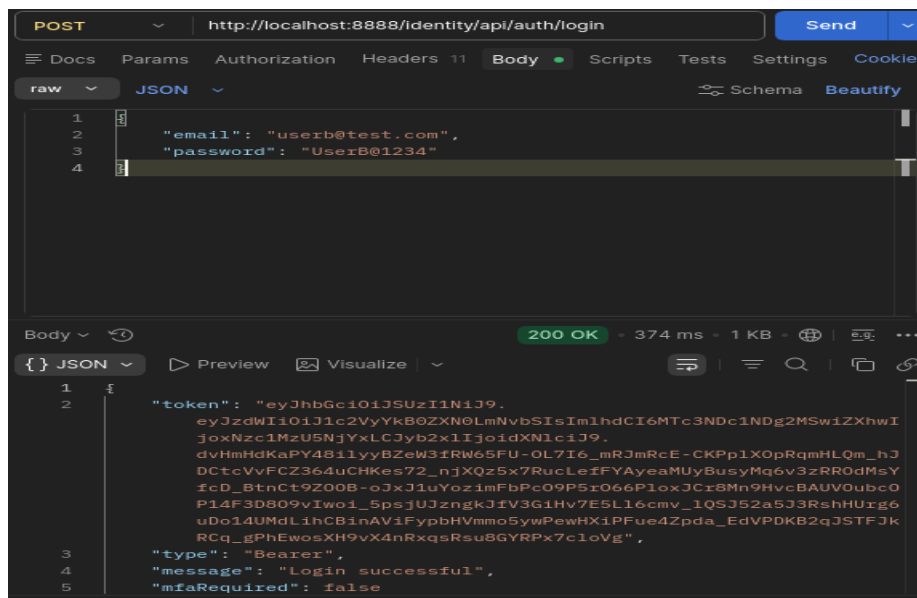
FINDING 05 — MFA NOT REQUIRED — SINGLE FACTOR AUTHENTICATION ONLY

API2:2023 — Broken Authentication

Endpoint	POST /identity/api/auth/login
Method	POST
OWASP API	API2:2023 — Broken Authentication
CVE Reference	CWE-308 (Use of Single-Factor Authentication) CWE-287 (Improper Authentication)
MITRE ATT&CK	T1078 — Valid Accounts T1556 — Modify Authentication Process
Severity	HIGH

Description

Every login response contains the field mfaRequired: false explicitly confirming that multi-factor authentication is not enforced for any user account. A single compromised password provides immediate full account access including vehicle location data and financial information.

Evidence

Login response for ALL accounts:

```
{ "token": "...", "type": "Bearer",
  "message": "Login successful", "mfaRequired": false }
```

CVE & MITRE Mapping

CWE-308	Use of Single-Factor Authentication — reliance on password alone as the only authentication mechanism for a platform handling location and financial data.
CWE-287	Improper Authentication — authentication process does not adequately verify claimed identity beyond a single factor.
T1078	MITRE ATT&CK: Valid Accounts — attacker uses compromised credentials for persistence and access without triggering MFA challenge.
T1556	MITRE ATT&CK: Modify Authentication Process — absence of MFA is effectively a weakened authentication process.

Remediation

- Implement TOTP-based MFA (Google Authenticator compatible) for all accounts
- Enforce MFA at account creation — not as optional setting
- Implement backup codes for account recovery
- Consider hardware key support (FIDO2/WebAuthn) for admin accounts

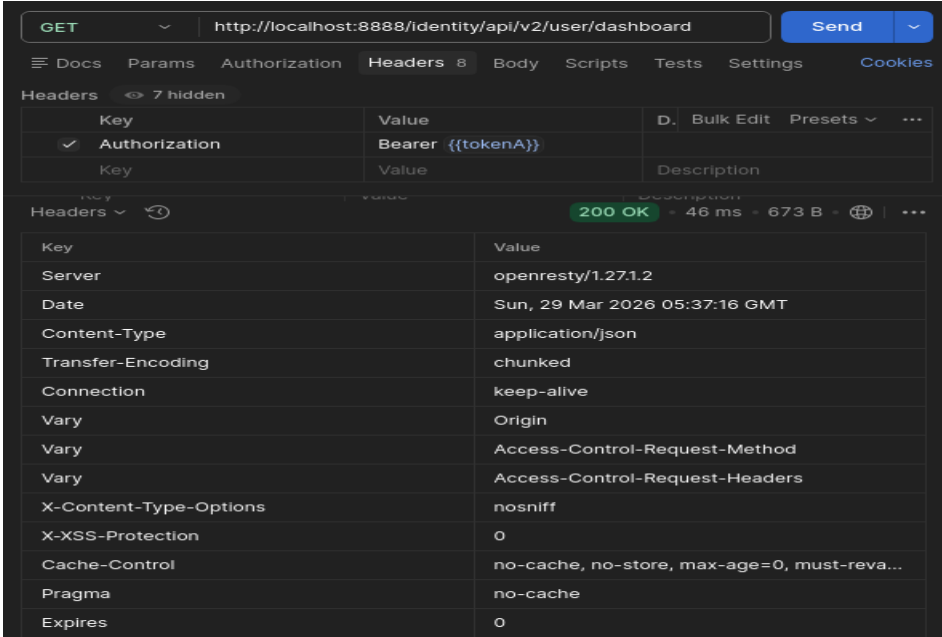
FINDING 06 — MISSING CONTENT-SECURITY-POLICY HEADER

API8:2023 — Security Misconfiguration

Endpoint	All API endpoints
Method	ALL
OWASP API	API8:2023 — Security Misconfiguration
CVE Reference	CVE-2016-5088 (Mozilla CSP Bypass) CWE-1021 (Improper Restriction of Rendered UI Layers)
MITRE ATT&CK	T1059.007 — JavaScript Execution T1185 — Browser Session Hijacking
Severity	HIGH

Description

All API endpoints are missing the Content-Security-Policy response header. Additionally X-XSS-Protection is set to 0 (disabled). Without CSP, cross-site scripting attacks can load external malicious scripts which execute in the victim's browser context, enabling session token theft and account takeover.

Evidence

Key	Value	Description
Authorization	Bearer {{tokenA}}	
Server	openresty/1.27.1.2	
Date	Sun, 29 Mar 2026 05:37:16 GMT	
Content-Type	application/json	
Transfer-Encoding	chunked	
Connection	keep-alive	
Vary	Origin	
Vary	Access-Control-Request-Method	
Vary	Access-Control-Request-Headers	
X-Content-Type-Options	nosniff	
X-XSS-Protection	0	
Cache-Control	no-cache, no-store, max-age=0, must-reva...	
Pragma	no-cache	
Expires	0	

Response headers from GET /identity/api/v2/user/dashboard:

X-Content-Type-Options: nosniff	✓ Present
X-Frame-Options: DENY	✓ Present
Strict-Transport-Security: ...	✓ Present
X-XSS-Protection: 0	X Disabled
Content-Security-Policy:	X MISSING

CVE & MITRE Mapping

CVE-2016-5088	CSP bypass in Mozilla — demonstrates how missing or misconfigured CSP enables script injection attacks that steal user data.
CWE-1021	Improper Restriction of Rendered UI Layers — without CSP, external scripts can render overlays to capture credentials.
T1059.007	MITRE ATT&CK: JavaScript Execution — injected JavaScript executes in browser context to steal JWT tokens from memory.
T1185	MITRE ATT&CK: Browser Session Hijacking — stolen session tokens allow attacker to impersonate victim.

Remediation

- Add Content-Security-Policy: default-src 'self' to all API responses
- Change X-XSS-Protection from 0 to 1; mode=block
- Implement CSP reporting to detect injection attempts: report-uri /csp-violation

FINDING 07 — CORS WILDCARD MISCONFIGURATION

API8:2023 — Security Misconfiguration

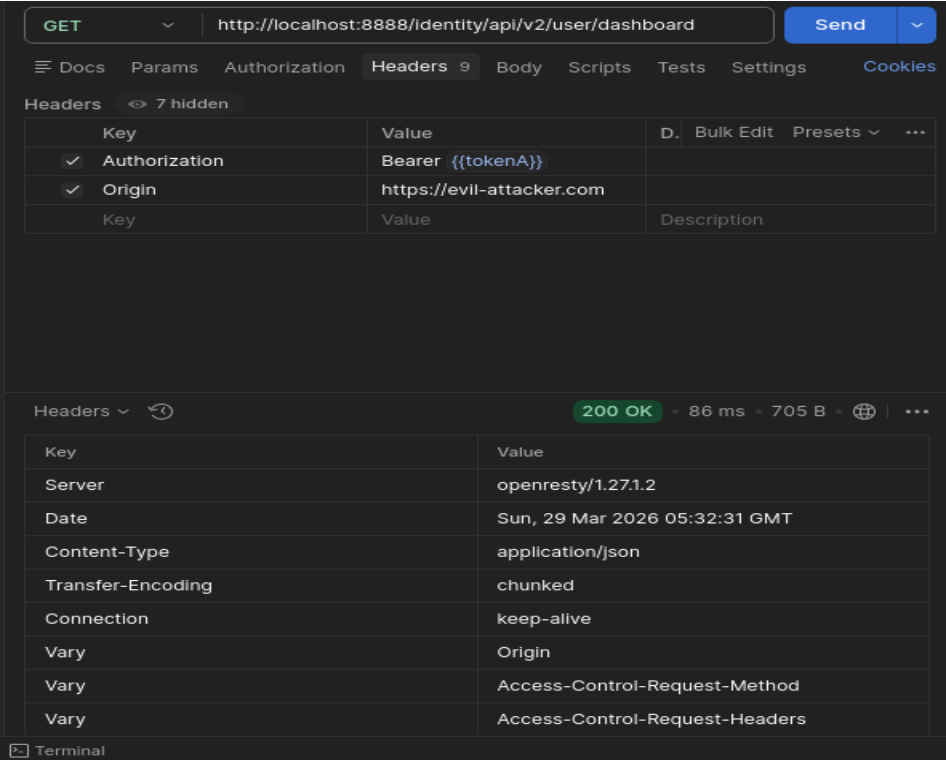
Endpoint	All API endpoints
Method	ALL
OWASP API	API8:2023 — Security Misconfiguration
CVE Reference	CVE-2018-1000007 (curl CORS Misconfiguration) CWE-942 (Overly Permissive Cross-Domain Whitelist)
MITRE ATT&CK	T1539 — Steal Web Session Cookie T1185 — Browser Session Hijacking
Severity	HIGH

Description

All API endpoints return Access-Control-Allow-Origin: * (wildcard) in response headers. This allows any website on the internet to make authenticated cross-origin requests to the crAPI backend. Testing with a forged Origin: https://evil-attacker.com header confirmed the server responds with data regardless of origin.

Evidence

Request with forged origin:
GET /identity/api/v2/user/dashboard
Origin: https://evil-attacker.com



The screenshot shows a web browser's developer tools with the Headers tab selected. The request is a GET to http://localhost:8888/identity/api/v2/user/dashboard. The request headers are: Authorization: Bearer {{tokenA}} and Origin: https://evil-attacker.com. The response is 200 OK, with headers: Server: openresty/1.27.1.2, Date: Sun, 29 Mar 2026 05:32:31 GMT, Content-Type: application/json, Transfer-Encoding: chunked, Connection: keep-alive, Vary: Origin, Vary: Access-Control-Request-Method, and Vary: Access-Control-Request-Headers.

Response:
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: POST, GET, OPTIONS, PUT, DELETE

Access-Control-Allow-Headers: Accept, Content-Type, Content-Length...
Status: 200 OK — full user data returned

CVE & MITRE Mapping

CVE-2018-1000007	curl CORS misconfiguration — credential forwarding across origins due to wildcard CORS policy. Direct parallel to this finding.
CWE-942	Overly Permissive Cross-Domain Whitelist — wildcard allows unrestricted cross-origin access to sensitive API resources.
T1539	MITRE ATT&CK: Steal Web Session Cookie — cross-origin attack reads session data from API responses via attacker-controlled website.
T1185	MITRE ATT&CK: Browser Session Hijacking — CORS wildcard enables cross-site request forgery and session theft.

Remediation

- Replace wildcard with explicit origin: Access-Control-Allow-Origin: http://localhost:8888
- Maintain a whitelist of allowed origins and validate against it dynamically
- Remove DELETE and PUT from allowed methods unless cross-origin write access is required
- Add Vary: Origin header to prevent caching of CORS responses

FINDING 08 — SERVER VERSION DISCLOSED IN RESPONSE HEADERS

API8:2023 — Security Misconfiguration

Endpoint	All API endpoints
Method	ALL
OWASP API	API8:2023 — Security Misconfiguration
CVE Reference	CWE-200 (Exposure of Sensitive Information) CWE-16 (Configuration)
MITRE ATT&CK	T1592 — Gather Victim Host Information T1590 — Gather Victim Network Information
Severity	LOW

Description

Every API response contains the header `Server: openresty/1.27.1.2` disclosing the server software name and exact version. Attackers use version information to search for published CVEs specific to that version and launch targeted exploits.

Evidence

The screenshot shows the 'Headers' tab of a web browser's developer tools. The request is a GET to `http://localhost:8888/identity/api/v2/user/dashboard`. The response status is 200 OK. The response headers are as follows:

Key	Value
Server	openresty/1.27.1.2
Date	Sun, 29 Mar 2026 05:32:31 GMT
Content-Type	application/json
Transfer-Encoding	chunked
Connection	keep-alive
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers

Server: openresty/1.27.1.2
(Present in every API response across all endpoints)

CVE & MITRE Mapping

CWE-200	Exposure of Sensitive Information to Unauthorised Actor — server version reveals attack surface details to potential adversaries.
T1592	MITRE ATT&CK: Gather Victim Host Information — attacker uses server header to identify software version and research known vulnerabilities.
T1590	MITRE ATT&CK: Gather Victim Network Information — version fingerprinting assists in infrastructure reconnaissance.

Remediation

- Configure openresty/nginx to suppress or genericize the Server header
- In nginx.conf add: server_tokens off;
- Optionally return: Server: webserver (generic non-identifying string)

FINDING 09 — WEAK EMAIL CHANGE TOKEN — NO EXPIRY**API2:2023 — Broken Authentication**

Endpoint	POST /identity/api/auth/change-email
Method	POST
OWASP API	API2:2023 — Broken Authentication
CVE Reference	CVE-2020-14179 (Weak Reset Token) CWE-640 (Weak Password Recovery Mechanism)
MITRE ATT&CK	T1110.003 — Password Spraying T1078 — Valid Accounts
Severity	HIGH

Description

The email change process sends a 20-character alphanumeric token to the user's registered email address. Analysis of the token via Mailhog confirms no expiry time is set — the token remains valid indefinitely. Combined with the absence of rate limiting (Finding 4), an attacker can brute force this token to take over any account.

Evidence

From: no-reply@example.com
Subject: **crAPI: Change Email Token**
To: userA@test.com

[Show headers ▼](#)

HTML [Plain text](#) [Source](#)

Hi,

We received a request to change your account email address. The previous e-mail address is: [usera@test.com](#) and the new one is: [userA@test.com](#)

To complete the process, please use the following token: **m1Q6v1m0t8k0i7b4Q8n9**

If you have not sent a request to change your email address, please ignore this message.

Thank You & have a wonderful day !

Warm Regards,
crAPI - Team
Email: support@crapi.io

This E-mail and any attachments are private, intended solely for the use of the addressee. If you are not the intended recipient, they have been sent to you in error: any use of information in them is strictly prohibited.

```
Email captured in Mailhog:
Subject: crAPI: Change Email Token
To: userA@test.com
Token: m1Q6v1m0t8k0i7b4Q8n9
Token length: 20 characters (alphanumeric only)
Expiry: NOT SET — token valid indefinitely
```

Attack Chain

This finding chains with Finding 4 (no rate limiting) to create a complete account takeover path:

- Step 1: Attacker requests email change for victim account
- Step 2: Token sent to victim email — attacker does not have access
- Step 3: Attacker brute forces token using automated tool (no rate limiting)
- Step 4: Token accepted — email changed to attacker-controlled address
- Step 5: Attacker requests password reset — sent to attacker email

- Step 6: Full account takeover — attacker logs in as victim

CVE & MITRE Mapping

CVE-2020-14179	Weak password reset token — insufficient entropy and no expiry allow token brute force. Direct parallel to this email change token finding.
CWE-640	Weak Password Recovery Mechanism for Forgotten Password — same weakness applies to email change tokens that are short and never expire.
T1110.003	MITRE ATT&CK: Password Spraying — brute forcing short tokens across the API endpoint with no lockout.
T1078	MITRE ATT&CK: Valid Accounts — successful token brute force results in attacker holding valid account credentials.

Remediation

- Use cryptographically secure random token generation — minimum 32 characters
- Set token expiry of 15 minutes maximum
- Invalidate token immediately after single use
- Rate limit token verification endpoint — maximum 3 attempts before token invalidation
- Alert user via their ORIGINAL email when an email change is requested

5. RISK CLASSIFICATION SUMMARY

#	Finding	OWASP	Severity	CVE / CWE
01	JWT Role in Payload + No MFA	API2:2023	HIGH	CWE-287, CWE-308
02	Excessive Data Exposure	API3:2023	MEDIUM	CWE-213
03	BOLA — GPS Location Leaked	API1:2023	CRITICAL	CVE-2021-20114, CWE-639
04	No Rate Limiting on Login	API4:2023	HIGH	CVE-2019-19781, CWE-307
05	MFA Not Required	API2:2023	HIGH	CWE-308
06	Missing CSP Header	API8:2023	HIGH	CVE-2016-5088, CWE-1021
07	CORS Wildcard (*)	API8:2023	HIGH	CVE-2018-1000007, CWE-942
08	Server Version Disclosed	API8:2023	LOW	CWE-200
09	Weak Email Token — No Expiry	API2:2023	HIGH	CVE-2020-14179, CWE-640

6. PREVENTION GUIDELINES

For API Developers

- Always verify object ownership server-side before returning any data — never trust client-supplied IDs
- Implement rate limiting on ALL authentication endpoints — use tools like nginx limit_req
- Enforce MFA for all accounts that access sensitive data (location, financial, PII)
- Apply strict CORS policy — whitelist specific trusted origins, never use wildcard
- Add Content-Security-Policy headers to all API responses
- Use cryptographically secure tokens with short expiry for all verification flows
- Filter API responses — return only the minimum fields necessary for each endpoint
- Suppress server version headers in all production environments

For Security Teams

- Include BOLA testing in all API security assessments — it is the #1 API risk worldwide
- Test CORS configuration by forging Origin headers in every API assessment
- Analyse JWT tokens on every engagement — decode payload, check algorithm, verify expiry
- Capture and analyse all email tokens via test mail servers — check entropy and expiry
- Map all findings to OWASP API Top 10, CVE, and MITRE ATT&CK for comprehensive reporting

7. CONCLUSION

This API Security Risk Analysis of OWASP crAPI identified 9 confirmed vulnerabilities spanning 4 OWASP API Security Top 10 categories. The most critical finding — Broken Object Level Authorization — presents a real-world physical safety risk by exposing victim GPS coordinates to any authenticated attacker. This goes beyond typical data breach scenarios and highlights why API security cannot be treated as an afterthought in modern SaaS platform development.

The combination of no rate limiting (Finding 4) and weak email change tokens (Finding 9) creates a complete account takeover chain requiring no sophisticated tooling — only patience and automation. The CORS wildcard misconfiguration (Finding 7) means these attacks can be launched from any website on the internet against any logged-in user.

Collectively these findings demonstrate that API security requires a fundamentally different approach to traditional web application security. The attack surface is larger, the data exposure is often invisible to end users, and the business impact — from GPS tracking to financial data exposure — is immediate and severe.

Immediate priority: Fix BOLA (Finding 3) and implement rate limiting (Finding 4). These two remediations eliminate the most critical physical safety risk and the primary account takeover vector.

Satheesh Nithiananthan (CyberLycan) | SAIZERO — Ground Zero Defence

Every shadow has a hunter.